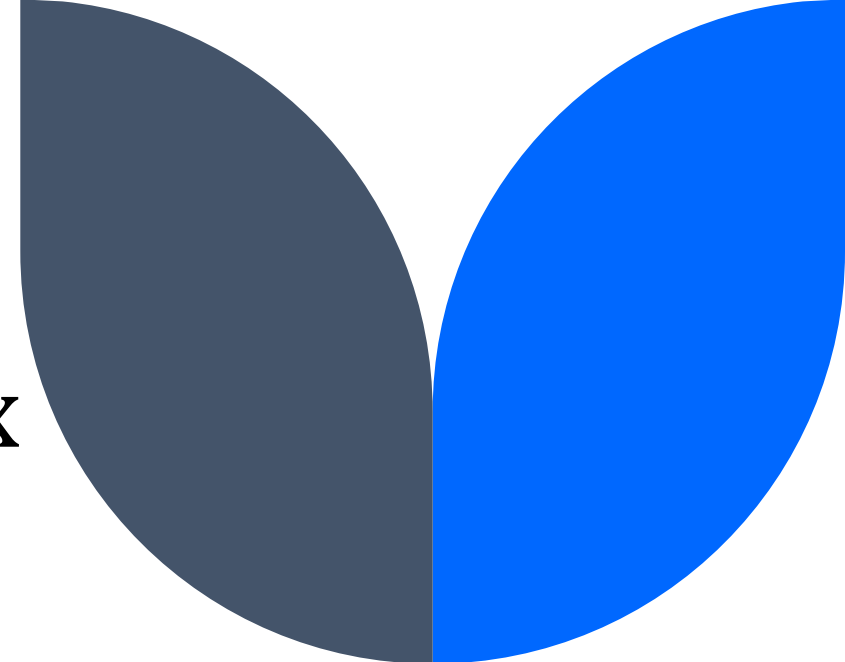
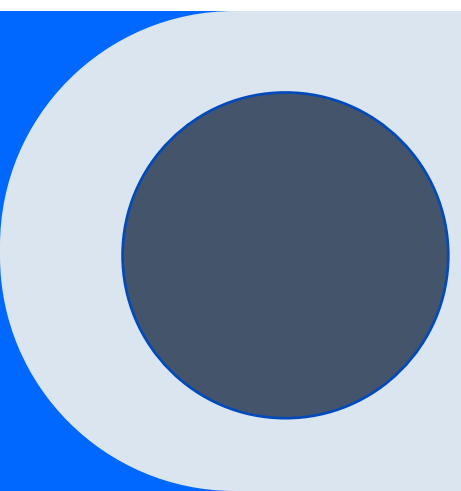




Аномалии в реляционных базах данных. Нормализация. Нормальные формы.

Валова Анастасия Александровна
НИТУ «МИСиС»



Аномалии в реляционных БД

Возникают при работе с отношениями, содержащими избыточные данные.

Виды аномалий:

- Аномалии удаления;
- Аномалии вставки;
- Аномалии обновления (модификации).

Перечисленных аномалий можно избежать путем **нормализации** исходного отношения.

Нормализация — это процесс организации данных в БД, включающий создание таблиц и установление отношений между ними в соответствии с правилами, которые обеспечивают защиту данных и делают базу данных более гибкой, устраняя избыточность и несогласованные зависимости.

Employee_ID	Name	Department	Student_group
123	Иванов И.И.	АСУ	БИВТ-23-1
234	Петров П.П.	АПД	БИВТ-23-2
234	Петров П.П.	АПД	БИВТ-23-3
456	Сидоров С.С.	БИСУП	БИВТ-23-4
456	Сидоров С.С.	БИСУП	БИВТ-23-1

Денормализованное отношение

ФИО студент	Сем	Дисциплина	ФО	О	КЧ	ФИО преподавателя
Петрова А.П.	1	Английский язык	зачет	1	60	Цветкова А.Ю.
Петрова А.П.	1	Мат. анализ	зачет	1	28	Рыбин К.К.
Петрова А.П.	1	Мат. анализ	экзамен	3	32	Раков И.И.
Петрова А.П.	1	Программирование	зачет	1	36	Незабудкина З.П.
Петрова А.П.	1	Программирование	экзамен	4	32	Зайчиков А.А.
Петрова А.П.	1	Линейная алгебра	зачет	1	24	Волков Г.И.
Петрова А.П.	1	Линейная алгебра	экзамен	4	28	Волков Г.И.
Петрова А.П.	1	История Отечества	экзамен	5	24	Москвин А.П.
Сидоров К.К.	3	Английский язык	зачет	1	60	Цветкова А.Ю.
Сидоров К.К.	3	Мат. анализ	зачет	1	20	Карпов К.Ю.
Сидоров К.К.	3	Мат. анализ	экзамен	5	28	Раков И.И.
Сидоров К.К.	3	Теория вероятностей и мат. статистика	экзамен	4	32	Соболев И.Г.
Сидоров К.К.	3	Операционные системы	зачет	1	36	Незабудкина З.П.
Сидоров К.К.	3	Операционные системы	экзамен	4	32	Незабудкина З.П.
Сидоров К.К.	3	Экономическая теория	зачет	1	24	Лабиринтов Е.Н.

Денормализованное отношение

ФИО студент	Сем	Дисциплина	ФО	О	КЧ	ФИО преподавателя
Петрова А.П.	1	Английский язык	зачет	1	60	Цветкова А.Ю.
Петрова А.П.	1	Мат. анализ	зачет	1	28	Рыбин К.К.
Петрова А.П.	1	Мат. анализ	экзамен	3	32	Раков И.И.
Петрова А.П.	1	Программирование	зачет	1	36	Незабудкина З.П.
Петрова А.П.	1	Программирование	экзамен	4	32	Зайчиков А.А.
Петрова А.П.	1	Линейная алгебра	зачет	1	24	Волков Г.И.
Петрова А.П.	1	Линейная алгебра	экзамен	4	28	Волков Г.И.
Петрова А.П.	1	История Отечества	экзамен	5	24	Москвин А.П.
Сидоров К.К.	3	Английский язык	зачет	1	60	Цветкова А.Ю.
Сидоров К.К.	3	Мат. анализ	зачет	1	20	Карпов К.Ю.
Сидоров К.К.	3	Мат. анализ	экзамен	5	28	Раков И.И.
Сидоров К.К.	3	Теория вероятностей и мат. статистика	экзамен	4	32	Соболев И.Г.
Сидоров К.К.	3	Операционные системы	зачет	1	36	Незабудкина З.П.
Сидоров К.К.	3	Операционные системы	экзамен	4	32	Незабудкина З.П.
Сидоров К.К.	3	Экономическая теория	зачет	1	24	Лабиринтов Е.Н.

Нормализация. Декомпозиция

Нормализация отношения - это декомпозиция исходного отношения на набор других отношений, обладающих меньшей степенью.

Декомпозиция позволяет эффективнее выполнять операции обновления, т.к. сокращается число проверок и вспомогательных действий, поддерживающих целостность БД.

Процесс декомпозиции – это замена отношения некоторым набором его проекций.

Декомпозиция должна быть обратимой, т. е. должна иметься возможность “собрать” исходное отношение из декомпозированных отношений без потери информации (декомпозиция без потерь).



Нормальные формы. 1НФ

Отношение находится *в первой нормальной форме (1НФ)* тогда и только тогда, когда схема этого отношения содержит только **простые** и только **однозначные атрибуты**, причем обязательно с одной и той же семантикой.

- Не допускается дублирование строк в таблице.

*Фирма	Модели
BMW	M5, X5M, M1
Nissan	GT-R



*Фирма	Модели
BMW	M5
BMW	X5M
BMW	M1
Nissan	GT-R

Составные атрибуты, в отличие от простых, – это атрибуты, составленные из нескольких простых атрибутов.

Многозначные атрибуты, в отличие от однозначных, – это атрибуты, представляющие множество значений.

Функциональная зависимость

Функциональная зависимость описывает связь между атрибутами отношения: если в отношении R , содержащем атрибуты A и B , атрибут B функционально зависит от атрибута A , то каждое отдельное значение атрибута A связано только с одним значением атрибута B (причем в качестве A и B могут выступать группы атрибутов).

A - **детерминант** функциональной зависимости.

$A \rightarrow B$  Сотрудник $\rightarrow$ Должность  «Иванов И.И.» $\rightarrow$ «Профессор»

$A \rightarrow B$ является **полной**, если удаление какого-либо атрибута из группы атрибутов A приводит к потере этой зависимости

$A \rightarrow B$ является **частичной**, если в группе атрибутов A есть один или несколько атрибутов, при удалении которых эта зависимость сохраняется

$A \rightarrow B, B \rightarrow C$ - атрибут C связан **транзитивной зависимостью** с атрибутом A через атрибут B

Нормальные формы. 2НФ

Отношение находится *во второй нормальной форме (2НФ)*, если оно удовлетворяет определению 1НФ и каждый неключевой атрибут функционально полно зависит от каждой части первичного ключа.

Неключевой атрибут – это любой атрибут отношения, не содержащийся в каком-либо первичном ключе или ключе-кандидате отношения.

Полная функциональная зависимость от ключа – это отсутствие функциональной зависимости от какой-либо части этого ключа.

Модель*	Фирма*	Цена	Скидка
M5	BMW	5500000	5%
X5M	BMW	6000000	5%
M1	BMW	2500000	5%
GT-R	Nissan	5000000	10%



Модель*	Фирма*	Цена
M5	BMW	5500000
X5M	BMW	6000000
M1	BMW	2500000
GT-R	Nissan	5000000

Фирма*	Скидка
BMW	5%
Nissan	10%

Нормальные формы. 3НФ

Отношение находится *в третьей нормальной форме (3НФ)*, если оно удовлетворяет определению 2НФ и каждый неключевой атрибут функционально полно зависит только от ключей (первичного и ключей кандидатов).

== каждый неключевой атрибут нетранзитивно функционально зависит от первичного ключа.

Не допускается наличие транзитивных функциональных зависимостей вида $A \rightarrow B$, $B \rightarrow C$.

Модель*	Автосалон	Телефон
BMW	Независимость	12-34-56
Audi	Независимость	12-34-56
Nissan	Genser	65-43-21



Модель*	Автосалон
BMW	Независимость
Audi	Независимость
Nissan	Genser

Автосалон*	Телефон
Независимость	12-34-56
Genser	65-43-21

Нормальные формы. НФ Бойса-Кодда

Отношение находится *в нормальной форме Бойса-Кодда (НФБК, или усиленная ЗНФ)* тогда и только тогда, когда она находится в ЗНФ и любая функциональная зависимость между ее атрибутами сводится к полной функциональной зависимости от *возможного* первичного ключа.

Отличие от ЗНФ: не только любой неключевой атрибут полностью функционально зависит от любого ключа, но и любой ключевой атрибут должен полностью функционально зависеть от любого ключа.

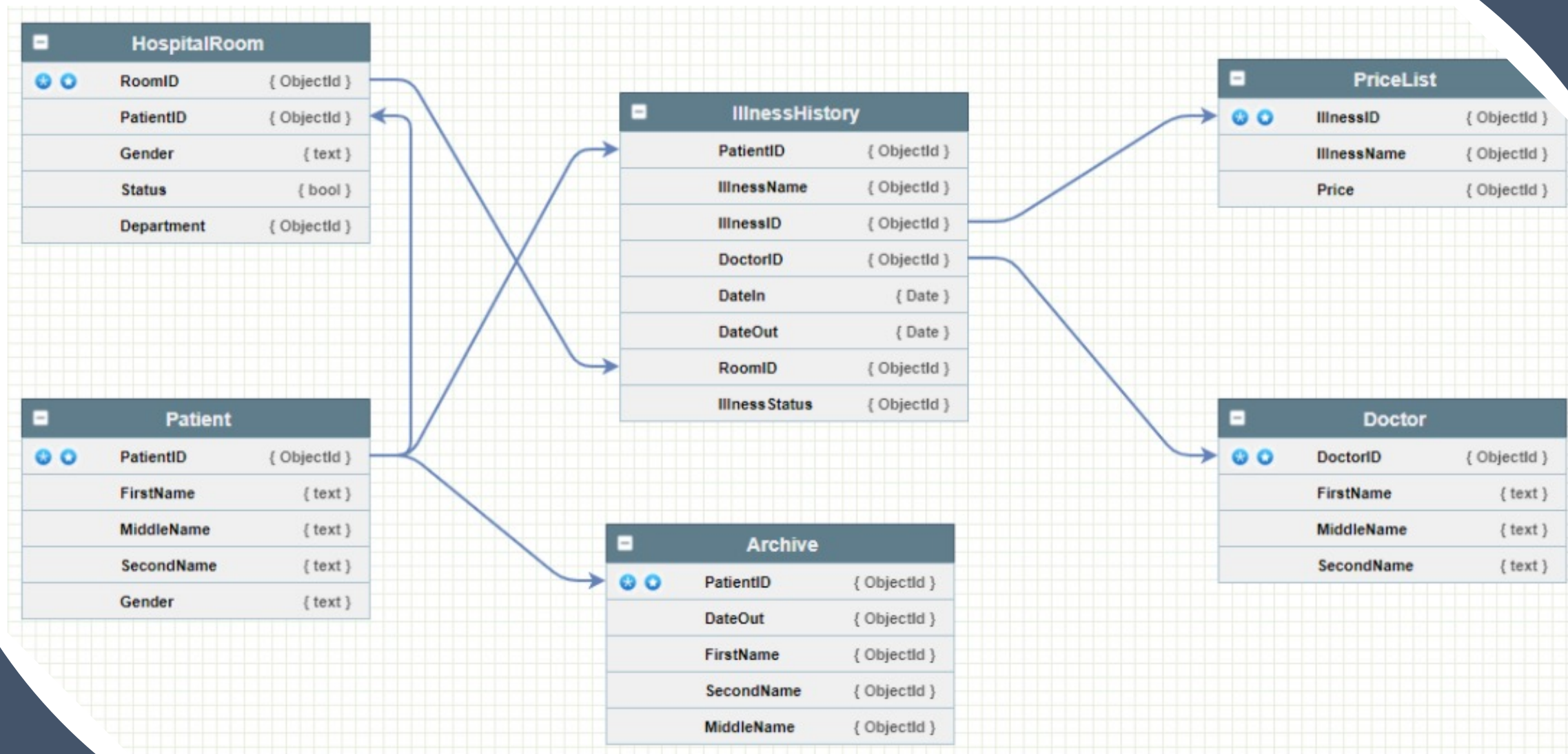
Стоянка	Начало	Окончание	Тариф
1	9:30	10:30	Бережливый
1	11:00	12:00	Бережливый
1	14:00	15:30	Стандарт
2	10:00	12:00	Премиум-В
2	12:00	14:00	Премиум-В
2	15:00	18:00	Премиум-А

Ключи: {Стоянка, Начало}, {Стоянка, Окончание}, {Тариф, Начало}, {Тариф, Окончание}.

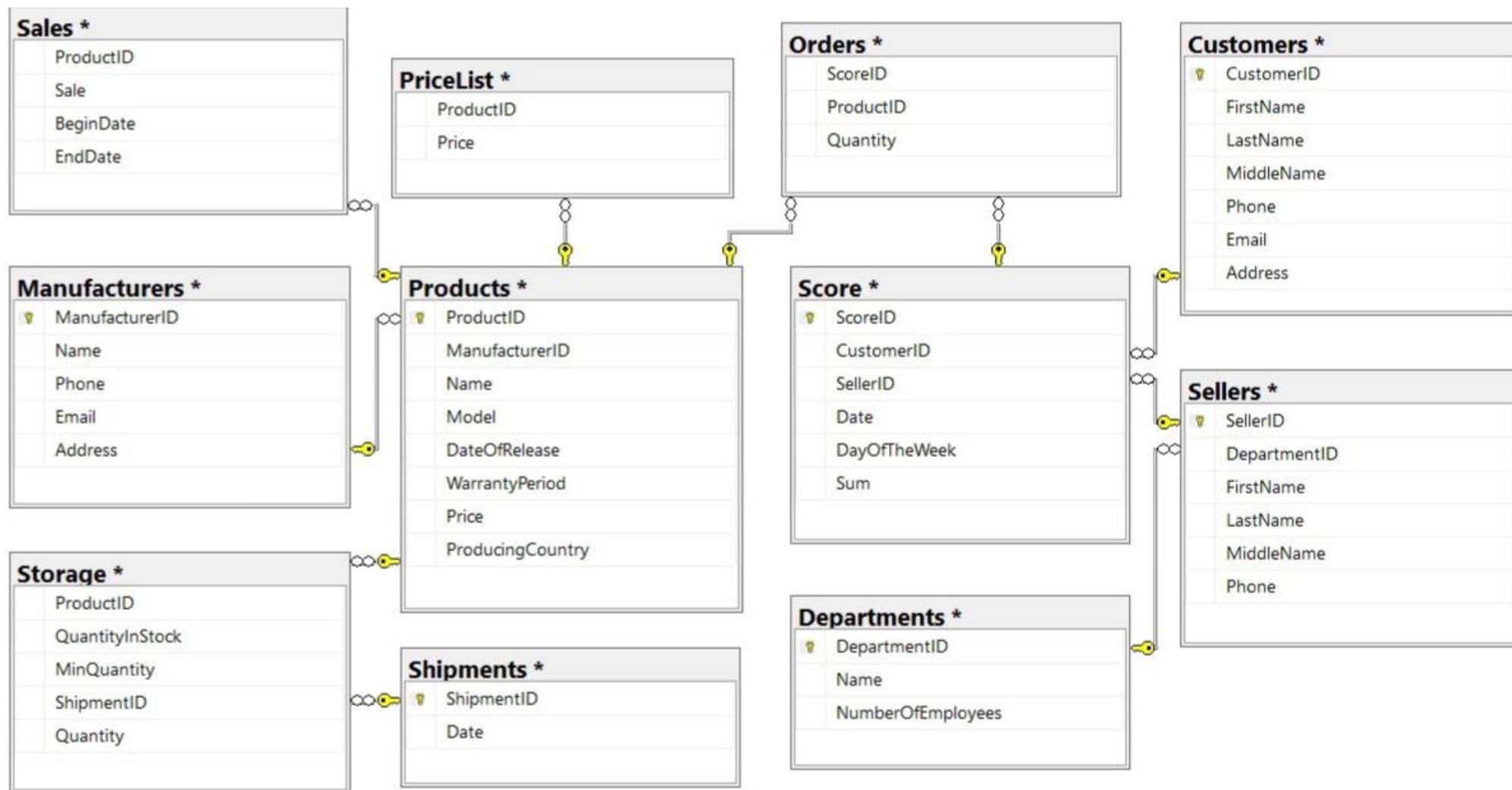
Тариф*	Стоянка
Бережливый	1
Стандарт	1
Премиум-А	2
Премиум-В	2

Тариф*	Начало*	Окончание
Бережливый	9:30	10:30
Бережливый	11:00	12:00
Стандарт	14:00	15:30
Премиум-В	10:00	12:00
Премиум-В	12:00	14:00
Премиум-А	15:00	18:00

Примеры ненормализованных БД



Примеры ненормализованных БД



9 Модификация данных

Вставка данных в таблицы

- **INSERT...VALUES**
 - Добавляет явно указанные значения.
 - Можно опустить значения в столбцах identity, допускающих NULL столбцах и столбцах с умалчиваемыми значениями.
 - Можно явно указать NULL и DEFAULT.
- **INSERT...SELECT / INSERT...EXEC**
 - Вставка в существующую таблицу результатов выполнения другого запроса или хранимой процедуры
- **SELECT...INTO**
 - Создает новую таблицу на основании результатов запроса
 - Не поддерживается в Azure SQL Database

Создание идентификаторов

Использование идентифицирующих столбцов

- Свойство столбца **IDENTITY** создает автоматически последовательность чисел для вставки в таблицу
 - Можно задавать начальное значение и приращение
 - Системные функции и переменные для определения вставленного значения:
 - ✓ **@@IDENTITY**: Последнее значение в сессии
 - ✓ **SCOPE_IDENTITY()**: Последнее значение в сессии и области видимости
 - ✓ **IDENT_CURRENT('<имя_таблицы>')**: Последнее значение в таблице

```
INSERT INTO Sales.Orders (CustomerID, Amount)
VALUES
(12, 2056.99);
...
SELECT SCOPE_IDENTITY() AS OrderID;
```

Создание идентификаторов

Использование последовательностей

- Последовательности – объекты, генерирующие последовательные числа
 - Поддержка в SQL Server 2012 и позднее
 - Существуют независимо от таблиц, поэтому гибче, чем identity
 - Используйте SELECT NEXT VALUE FOR для извлечения следующего числа в последовательности
 - Можно установить в качестве значения по умолчанию для столбца

```
CREATE SEQUENCE Sales.OrderNumbers AS INT  
START WITH 1 INCREMENT BY 1;  
...  
SELECT NEXT VALUE FOR Sales.OrderNumbers;
```


Обновление данных в таблице

Оператор UPDATE

- Обновляет все строки в таблице или представлении
 - Фильтрация строк для установки значений через предложение WHERE
 - Устанавливаемые значения можно определять из предложения FROM
- Модифицируются только столбцы, указанные в предложении SET

```
UPDATE Production.Product  
SET unitprice = (unitprice * 1.04)  
WHERE categoryid = 1 AND discontinued = 0;
```

Обновление данных в таблице

Оператор MERGE

MERGE модифицирует данные на основе условия

- Когда источник соответствует цели
- Когда источник не имеет соответствия с целью
- Когда цель не имеет соответствия с источником

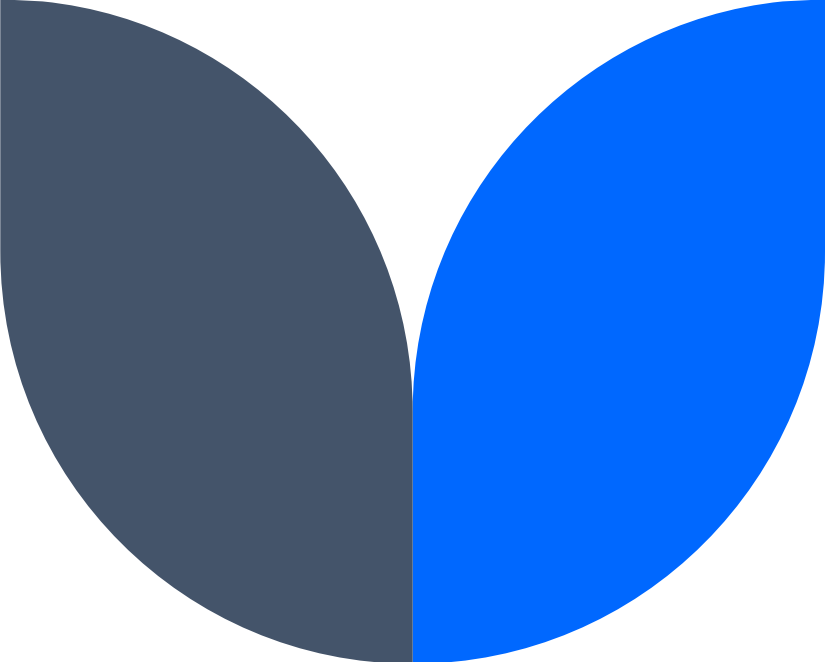
```
MERGE INTO Production.Products as P                                -- цель
      USING Production.ProductsStaging as S ON P.ProductID=S.ProductID -- источник
WHEN MATCHED THEN
  UPDATE SET
    P.UnitPrice = S.UnitPrice, P.Discontinued=S.Discontinued
WHEN NOT MATCHED [BY TARGET] THEN
  INSERT (ProductName, CategoryID, UnitPrice, Discontinued)
  VALUES (S.ProductName, S.CategoryID, S.UnitPrice, S.Discontinued)
WHEN NOT MATCHED BY SOURCE THEN
  DELETE;
```

Удаление данных из таблицы

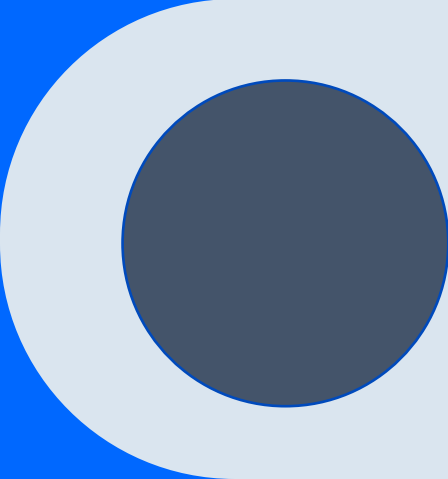
- DELETE без предложения WHERE удаляет все строки
 - Используйте предложение WHERE для отбора удаляемых строк

```
DELETE FROM SalesLT.SalesOrderDetail  
WHERE SalesOrderID = 10248;
```

- TRUNCATE TABLE очищает всю таблицу
 - Хранилище физически очищается, строки не удаляются по отдельности
 - Минимальное журналирование (сбрасывает identity, триггер не активируется)
 - Можно откатить, если TRUNCATE выполняется в транзакции
 - TRUNCATE TABLE потерпит неудачу, если на таблицу указывает внешний ключ из другой таблицы



10 Процедурное программирование на Transact-SQL



Переменные

- Переменные – это объекты, которые позволяют сохранять значение для последующего использования в том же пакете
- Переменные объявляются с использованием оператора DECLARE
 - Переменные могут быть объявлены и инициализированы в том же операторе
- Переменные всегда локальны для пакета в котором они объявлены и выходят за область видимости при завершении пакета

```
-- Объявление и инициализация переменных  
DECLARE @color nvarchar(15)='Black', @size nvarchar(5)='L';  
-- Использование переменных в операторах SQL  
SELECT *  
FROM Production.Product  
WHERE Color=@color and Size=@size;  
GO
```

Условное ветвление

- IF...ELSE использует предикат для определения пути выполнения кода
 - Код в блоке IF выполняется, если предикат разрешается в TRUE
 - Код в блоке ELSE выполняется, если предикат разрешается в FALSE или UNKNOWN

```
IF @color IS NULL
    SELECT * FROM Production.Product;
ELSE
    SELECT * FROM Production.Product
    WHERE Color = @color;
```

```
IF @color IS NULL
begin
    SELECT * FROM Production.Product;
end
ELSE
begin
    SELECT * FROM Production.Product
    WHERE Color = @color;
end
```

Циклы

- WHILE позволяет выполнять код в цикле
- Операторы в блоке WHILE повторяются, пока предикат вычисляется в TRUE
- Цикл заканчивается, когда предикат вычисляется в FALSE или UNKNOWN
- Выполнение можно изменить через BREAK или CONTINUE

```
DECLARE @custid AS INT=1, @lname AS NVARCHAR(20);
WHILE @custid <=5
BEGIN
    SELECT @lname = lastname FROM Sales.Customer
    WHERE customerid = @custid;
    PRINT @lname;
    SET @custid += 1;
END;
```

Хранимые процедуры. Типы

- Объекты БД, инкапсулирующие выполняемый на сервере код
- Системные хранимые процедуры:
 - Имя начинается с **sp_**
 - Созданы в БД master
 - Предназначены для применения в любой БД
 - Часто используются системными администраторами
- Пользовательские хранимые процедуры :
 - Определены в локальной БД
 - Имя часто начинается с префикса **pr_**
 - **Типы:** Transact-SQL и CLR (замена для Extended SP)

Хранимые процедуры (Stored Procedures)

- Могут быть параметризованы (@param as *datatype* = *default* [OUT])
 - Входные параметры
 - Output parameters

```
CREATE PROCEDURE SalesLT.GetProductsByCategory (@CategoryID INT = NULL)
AS
IF @CategoryID IS NULL
    SELECT ProductID, Name, Color, Size, ListPrice
    FROM SalesLT.Product
ELSE
    SELECT ProductID, Name, Color, Size, ListPrice
    FROM SalesLT.Product
    WHERE ProductCategoryID = @CategoryID;
```

- Учитывается порядок параметров

Подсказка: Размещайте параметры с умалчиваемыми значениями в конце списка параметров (для гибкости)

Вызов хранимых процедур с параметрами

```
CREATE PROCEDURE dbo.pr_GetTopProducts  
(@StartID as int = 1, @EndID as int = 20) AS ...
```

- По имени:
 - EXEC pr_GetTopProducts
 - @StartID = 1, @EndID = 10
- По позиции:
 - EXEC pr_GetTopProducts 1, 10
- Использование умалчиваемых значений
 - EXEC pr_GetTopProducts @EndID=10

Хранимые процедуры (SP). Некоторые соглашения

Выходные параметры:

- Используются для возврата клиенту не табличных значений
- При вставке записи возвращать значение поля-счетчика

Проверка ошибок и валидация данных:

- Модифицирующие данные процедуры могут выполнять проверку данных

RETURN (int):

- Используется для возврата признака успеха/неудачи
- Оператор RETURN завершает выполнение процедуры